

CloudX Memory Plugin Guide

How the Documentation Archive plugin stores, searches, invalidates, and ports local knowledge

CloudX

2026-06-07

Table of contents

1	Executive Summary	2
2	Big Picture	2
3	Components	3
3.1	CloudX Plugin	3
3.2	HTTP Client	3
3.3	AI Enrichment Service	3
3.4	Web Panel	3
3.5	Indexer API	3
3.6	Archive Engine	4
4	Archive Layout	4
5	Ingestion Flow	4
5.1	Browser Upload Ingest	5
5.2	Local Path Ingest	5
5.3	URL Ingest	6
5.4	Text Ingest	7
5.5	Extraction And Chunking	7
5.6	AI Enrichment	8
6	Storage Model	9
7	Search Flow	10
8	State And Invalidation	10
9	API And Hook Map	11
9.1	Read Hooks	11
9.2	Write And External Hooks	11
10	UI Workflow	12
11	Default Rule And Skills	13

12 Setup And Operations	14
13 Rendering This PDF	16
14 What Is Tested	16
15 Current Boundaries	17
16 Source Inspection Commands	17

1 Executive Summary

The new memory plugin is implemented as the CloudX `documentation` plugin plus a local FastAPI service called the Documentation Indexer. Its job is to turn uploaded files, allowed local files and directories, URLs, YouTube videos/playlists, copied text, and transcripts into a portable local knowledge archive that CloudX can search later through plugin hooks, the Documentation tab, HTTP, automation, and voice-aware read paths.

The system is intentionally local-first. The archive lives under `CLOUDX_DOCUMENTATION_DATA_DIR`, defaulting to `.cloudx/documentation`. That directory contains the SQLite catalog, immutable source snapshots, and the Turbovec dense index. First-class archive export/import packages that directory as a validated ZIP with an SQLite online-backup snapshot of `catalog.sqlite`, file hashes, and explicit merge or replace flows.

The retrieval model is hybrid by default: lexical SQLite FTS5 search and a local hash-based dense vector search are combined with reciprocal rank fusion. Non-active documents remain auditable, but the dense index is rebuilt only from active chunks.

2 Big Picture

The request path has six layers:

1. The browser panel, plugin hooks, HTTP routes, automation, or a Codex skill creates an ingest, search, invalidate, or maintenance request.
2. The CloudX server plugin validates CloudX concerns such as allowed local paths, hook safety, and browser upload limits.
3. `DocumentationClient` forwards the request to the local Documentation Indexer.
4. The indexer extracts content, stores source snapshots, updates SQLite and FTS5, and rebuilds the Turbovec index when active chunks change.
5. If Documentation AI enrichment is enabled in Settings and global AI control is on, the server loads the configured documentation enrichment skills, calls the same Codex exec model used by voice control, and writes derived AI chunks back through the indexer.
6. Codex tabs receive the documentation system rule and skills automatically through the CloudX overlay path used by built-in rules and skills.

The browser UI and other CloudX surfaces do not directly mutate archive files. They call plugin hooks such as `documentation.search`, `documentation.ingest.path`, and `documentation.invalidate`. The server-side plugin resolves allowed local paths and forwards requests through `DocumentationClient` to the local indexer.

The indexer owns all persistence. It initializes the archive directories, extracts text, writes source snapshots, stores document and chunk metadata in SQLite, updates FTS5 rows through SQLite triggers, and rebuilds the Turbovec file after ingest or invalidation.

3 Components

3.1 CloudX Plugin

Source: `apps/server/src/plugins/DocumentationPlugin.ts`

Registers the `documentation` plugin, exposes hooks, contributes the UI renderer, enforces path policy for local path ingest, and declares default documentation rules and skills as automatic CloudX system contributions.

3.2 HTTP Client

Source: `apps/server/src/documentation/DocumentationClient.ts`

Wraps the indexer REST API, applies request timeout, bounds response size, and normalizes service errors.

3.3 AI Enrichment Service

Source: `apps/server/src/documentation/DocumentationEnrichmentService.ts`

Runs only when the `documentation.aiEnrichmentEnabled` plugin setting is true and global AI control is enabled. It reads the configured CloudX skills from the rules/skills catalog, builds complete evidence batches from all source-origin chunks, extracted table/figure/image/schematic artifact manifests, optional ASR transcript segments, and video keyframe paths, then invokes Codex exec with a strict JSON schema for each batch. Returned spans are merged and written as AI-origin chunks by `POST /documents/{id}/enrich`.

3.4 Web Panel

Source: `apps/web/src/ui/DocumentationPanel.tsx`

Provides search, filters, ingest forms, invalidation buttons, manifest view, archive export/import controls, and index rebuild controls.

3.5 Indexer API

Source: `services/documentation-indexer/src/cloudx_documentation_indexer/main.py`

Defines FastAPI routes for health, stats, documents, ingest, enrichment, search, invalidate, delete, rebuild, archive export, and archive import.

3.6 Archive Engine

Source: `services/documentation-indexer/src/cloudx_documentation_indexer/archive.py`

Stores snapshots and metadata, extracts text, chunks content, embeds chunks, builds the Turbovec index, searches, and records invalidation history.

4 Archive Layout

By default, the archive root is `.cloudx/documentation`. It can be changed with `CLOUDX_DOCUMENTATION_DATA_DIR`.

The archive root contains:

Path	Purpose
<code>catalog.sqlite</code>	SQLite catalog for documents, chunks, AI enrichment records, FTS5 search rows, and invalidation events.
<code>snapshots/<sha256>/...</code>	Immutable source bytes captured at ingest time. URL snapshots may also include <code>metadata.json</code> with final URL, content type, ETag, and last-modified headers. Rich extraction sidecars live under <code>snapshots/<sha256>/extracted/</code> .
<code>indexes/local-hash-64/chunks.tvim</code>	Turbovec dense index built from active chunks only.
<code>indexes/local-hash-64/manifest.json</code>	Rebuild metadata for schema version, embedding profile, Turbovec version, index format, dense threshold, and active chunk count.

The portable manifest endpoint walks the archive root and returns each file path, logical byte size, allocated byte size when the platform reports it, category, and SHA-256. It also returns `archiveSize` totals for logical bytes, allocated disk bytes, database bytes, source snapshot bytes, extracted artifact bytes, index bytes, and the dense-index runtime estimate. Archive export creates a ZIP package with that same archive root, a manifest with file hashes and schema/profile metadata, and a live `catalog.sqlite` snapshot produced through SQLite online backup semantics. Manual root copies are still possible for low-level maintenance, but only after writes are stopped and only as a complete archive directory.

5 Ingestion Flow

The plugin supports four ingest paths.

5.1 Browser Upload Ingest

The Documentation panel defaults to upload mode. The browser sends file bytes to `POST /api/documentation/upload`; the CloudX server enforces the configured upload cap and forwards the bytes to the indexer's multipart `POST /ingest/upload` route.

Upload ingest stores the original filename and content type in snapshot metadata, infers a source type from the filename and content type, autodetects the title from the filename when the title field is blank, assigns the default `uploads` collection when collection is blank, and records a stable `upload://<safe-name>` URI. This path is the preferred user workflow for datasheets, PDFs, spreadsheets, images, Markdown, copied exports, and other files that are on the user's workstation but not necessarily visible as a local path to the indexer process.

When AI enrichment is enabled, browser upload is also the path that can pass media bytes to the enrichment service. Uploaded audio/video can be transcribed through the existing Faster Whisper ASR service, and uploaded video is converted into a scene-selected keyframe manifest with FFmpeg before Codex is asked to improve the import. Long media files can still produce large keyframe manifests, so the enrichment service batches the manifest instead of silently capping it.

5.2 Local Path Ingest

`documentation.ingest.path` accepts a file or directory. Before the request reaches the indexer, CloudX resolves the path through `PathPolicy`, so the file must be under configured allowed roots. When the bundled helper runs with `CLOUDX_SERVER_URL`, relative paths resolve from the helper process's current workspace. Direct raw indexer calls through `CLOUDX_DOCUMENTATION_URL` require absolute paths. If a directory is passed, the indexer recursively ingests supported documentation file suffixes. Code-heavy files or directories require `acceptGeneratedCodeDocumentation: true`; otherwise the request fails before writing archive documents.

Supported suffixes include Markdown, text, PDF, HTML, JSON, YAML, XML, CSV, XLS/XLSX workbooks (`.xls`, `.xlsx`, `.xslm`, `.xlsb`, `.ods`, `.ots`), SRT/VTT, Python, TypeScript, JavaScript, C/C++, Rust, CSS, AsciiDoc, LaTeX, and images (`.png`, `.jpg`, `.jpeg`, `.gif`, `.tif`, `.tiff`, `.bmp`, `.webp`). Python, TypeScript, JavaScript, C/C++, and Rust source files are documentation-first inputs: the indexer stores generated Markdown as the searchable document and does not index raw source bodies.

The supported source-type labels are `datasheet`, `book`, `website`, `readme`, `media`, `image`, `spreadsheet`, and `text`; generated code documentation is stored with source type `repo_code`. Source type describes the source for hook/API filters and skill behavior; extraction is selected from the actual file suffix, content type, and file signature. That means setting `sourceType` to `datasheet` no longer forces a Markdown, text, spreadsheet, or image file through the PDF extractor.

When title is blank, path ingest uses the file name. When collection is blank, directory ingest uses the directory name for every file under that ingest request and single-file ingest uses the parent folder name.

Input class	Examples	Extraction behavior
PDF	<code>.pdf</code> , <code>application/pdf</code> , %PDF-bytes	Page text, tables, rendered visual page artifacts, and Phase 1 schematic description artifacts for schematic-like pages.
Image	<code>.png</code> , <code>.jpg</code> , <code>.jpeg</code> , <code>.gif</code> , <code>.tif</code> , <code>.tiff</code> , <code>.bmp</code> , <code>.webp</code> , <code>image/*</code>	Normalized PNG artifacts for all frames plus format, size, mode, frame metadata, and Phase 1 schematic description artifacts for schematic-like images.
Spreadsheet	<code>.xls</code> , <code>.xlsx</code> , <code>.xslm</code> , <code>.xlsb</code> , <code>.ods</code> , <code>.ots</code> , Excel/OpenDocument spreadsheet content types	One searchable chunk per sheet plus portable CSV, Markdown, JSON, and <code>spreadsheet_index.tsv</code> artifacts. XLSX/XLSM formulas and merged ranges are preserved as metadata where the file exposes them.
HTML	<code>.html</code> , <code>.htm</code> , <code>text/html</code>	Text after removing script, style, template, and noscript content.
Text	<code>.md</code> , <code>.txt</code> , <code>.json</code> , <code>.yaml</code> , <code>.xml</code> , <code>.csv</code> , <code>.css</code> , <code>.adoc</code> , <code>.tex</code>	UTF-8 text decode with replacement for invalid bytes.
Vendor code	<code>.py</code> , <code>.ts</code> , <code>.tsx</code> , <code>.js</code> , <code>.c</code> , <code>.cpp</code> , <code>.h</code> , <code>.hpp</code> , <code>.rs</code>	Requires generated-code documentation review. The searchable snapshot is generated Markdown with file hashes, symbol summaries, imports, call-flow cues, configuration/hardware tokens, and integration hazards. <code>extracted/vendor_code/code_manifest.js</code> records covered files and parser status. Raw source artifacts are retained only when <code>retainRawCodeArtifacts: true</code> is explicitly supplied.
Captions and transcripts	<code>.srt</code> , <code>.vtt</code> , manually supplied transcript text	Text chunks marked as media when requested.

5.3 URL Ingest

`documentation.ingest.url` downloads a URL with redirects enabled and a 20 second timeout. The indexer records the original URL, final URL, content type, ETag, and last-modified header when those values are available.

When title is blank, URL ingest uses the URL leaf or a YouTube video/playlist label. When collection is blank, normal URL ingest uses the URL host and YouTube ingest uses `youtube` unless the URL is a playlist.

If the caller provides a transcript, the URL is ingested as media text without downloading page content. If the source type is `media` or the URL is recognized as a single YouTube video, the indexer reads video metadata with `yt-dlp`, stores the video description as its own searchable source chunk, and runs timestamped ASR transcript extraction alongside local slide-frame scanning. The visual path downloads one bounded-resolution video source, FFmpeg scans local video segments in parallel, and deterministic image-difference checks preserve one selected JPEG frame per detected slide or visual state under the source snapshot's `extracted/media/keyframes/` directory. After ASR and visual scan both finish, selected frames are aligned to transcript timestamps. The keyframe index, timestamped transcript segment index, visual sampling manifest, YouTube metadata JSON, optional `description.txt`, description chunks, transcript chunks, and original URL are all stored as source evidence. The default ASR backend is faster-whisper. `CLOUDX_DOCUMENTATION_ASR_BACKEND=whisper-cpp` selects a compiled `whisper.cpp` CLI backend, which requires an explicit `CLOUDX_DOCUMENTATION_WHISPER_CPP_BIN` and `CLOUDX_DOCUMENTATION_WHISPER_CPP_MODEL_PATH`. When `CLOUDX_DOCUMENTATION_WHISPER_CPP_VAD=true`, it also requires `CLOUDX_DOCUMENTATION_WHISPER_CPP_VAD_MODEL_PATH`; the installer configures Silero VAD so silent windows are skipped instead of decoded into repeated filler text. Missing `yt-dlp`, missing `ffmpeg`, missing backend dependencies, transcript failures, or keyframe extraction failures fail the ingest request explicitly.

If the URL contains a YouTube `list=` playlist ID and no manual transcript is supplied, `/ingest/url` treats it as a playlist. The indexer uses `yt-dlp` in flat playlist mode to read playlist metadata, then ingests each video through the same single-video path. Each playlist entry becomes one archive document: the document title comes from the video entry title, the URI is the canonical `https://www.youtube.com/watch?v=<video-id>` URL, the source type is `media`, and the collection is the playlist title unless the caller provided an explicit collection. Transcript text comes from faster-whisper by default, and every successfully ingested playlist entry gets its own selected slide-frame manifest. If a later playlist entry fails, earlier entries from that request are marked deleted so active search does not contain a partial playlist import.

YouTube enrichment follows the same document boundary. A single video URL enriches the media document that was created for that video. A playlist URL expands first, then each playlist entry is enriched independently using that entry's description chunk, transcript chunks, metadata, and keyframe artifact paths.

5.4 Text Ingest

`documentation.ingest.text` stores direct text, copied source material, or manually supplied transcripts. It requires only non-empty text. It rejects `repo_code` and code-like URI suffixes because pasted raw source has no file provenance for generated documentation or manifest coverage. If title is blank, the indexer uses the first non-empty text line. If URI is blank, it creates a deterministic `manual://<title>-<hash>` URI. If source type is blank, it infers one from the URI and otherwise uses `text`. If collection is blank, it uses the URI scheme such as `manual` or the URL host when an HTTP URI is supplied.

5.5 Extraction And Chunking

PDF input is extracted with a table-aware pipeline. Page text uses `page N` locators; detected tables become searchable Markdown chunks and portable sidecars under `extracted/tables/`; pages with visual objects are rendered to PNG sidecars under `extracted/figures/` so graphs, plots, flowcharts, schematics, and layout-heavy pages can be inspected later. Schematic-like PDF pages and standalone images get Phase 1 schematic records under `extracted/schematics/<id>/` with a searchable source chunk,

saved description Markdown, JSON analysis metadata, a pointer back to the exact rendered/original image artifact, deterministic classification reasons, reference designator and net-label candidates from existing text or filenames, connection-cue metrics, and `analysisOutputs: []` for future structured analyzers. Spreadsheet input creates one source chunk per sheet with `sheet <name> range <A1:...>` locators and writes CSV, Markdown, JSON, and manifest sidecars under `extracted/spreadsheets/`. XLSX/XLSM sheets retain formula text and merged-range metadata; legacy XLS values are read through the XLS engine. Vendor code input generates searchable Markdown with `code-doc ...` locators and writes `extracted/vendor_code/code_manifest.json`; raw source code is not indexed and is retained as artifacts only when explicitly requested. Image input is normalized to PNG sidecars under `extracted/images/`; multi-frame images preserve every frame as a separate artifact and are indexed with format, size, mode, frame count, and visual-artifact metadata. HTML input is parsed with script, style, template, and noscript content removed. Other input is decoded as text. Extracted spans are split into paragraph-like chunks with a maximum target size of 1200 characters.

This is similar to the datasheet-analysis workflow in the important ways for storage and recall: tables become standalone artifacts, visual pages are preserved as rendered images, and each chunk carries a locator. It is not OCR. Text inside a graph or scanned page is searchable only when it is present in the PDF text layer or table extraction output; otherwise the graph or flowchart is preserved as an artifact for inspection and citation.

Each document ID is deterministic for a (`uri`, `content_sha256`) pair. Re-ingesting the same URI and content replaces its chunks and keeps the document active.

5.6 AI Enrichment

AI enrichment is a plugin-wide setting in Settings under Documentation:

Setting	Default	Meaning
<code>aiEnrichmentEnabled</code>	<code>true</code>	Enables the post-ingest AI pass and assisted answers when global AI control is also enabled.
<code>aiEnrichmentSkillIds</code>	<code>documentation-enrich-metadata,documentation-enrich-visuals,documentation-enrich-media</code>	Comma-separated CloudX skill ids that define what the AI should extract and how it should write derived spans.

There is no per-upload hardcoded extraction recipe. The service loads the configured skills from the same folder-backed rules/skills catalog used by `create-cloudx-skill`, includes those instructions in the Codex prompt, and requires Codex to return structured JSON with `summary`, `spans`, `metadata`, and `warnings`.

The AI pass runs after the source import succeeds. It reads the newly ingested document, all source-origin chunks, portable extraction artifact manifests, and media evidence when available. For media uploads, the existing ASR service produces a timestamped transcript and FFmpeg produces scene-selected keyframe artifacts. For YouTube URL imports, the indexer itself stores timestamped faster-whisper transcript segments, metadata, selected slide-frame artifacts, and a visual sampling manifest before enrichment runs. The service batches all chunks, artifact paths, transcript segments, and keyframe paths so prompt size is controlled without dropping later evidence. Codex returns derived spans with locators such as

`ai:metadata`, `ai:visual`, and `ai:media`; the indexer stores them as `chunk_origin = 'ai'` so search can distinguish source extraction from AI-derived enrichment. Re-running enrichment for a document replaces previous AI chunks for that document and leaves source chunks unchanged.

If enrichment fails, the base source document remains active. The ingest response includes an `enrichment` status block with the affected document id and error message.

The same setting also controls assisted archive answers in the browser panel. When AI assistance is available, the Documentation panel's answer mode calls the `documentation.answer` hook. That hook searches the archive, opens matching source documents, sends selected source evidence to Codex with a strict answer schema, and returns plaintext, sanitized-renderable semantic HTML, citations, warnings, and the underlying results. The panel shows an explicit running state while this search and answer pass is active. When AI assistance is disabled, the answer mode is unavailable; CloudX falls back to manual search and source viewing only.

6 Storage Model

SQLite has four durable tables and one FTS5 virtual table:

Store	Meaning
<code>documents</code>	One row per ingested source, including title, source type, URI, snapshot path, content hash, state, collection, tags, and timestamps.
<code>chunks</code>	Extracted and AI-derived chunks with locators, state, <code>chunk_origin</code> , and optional <code>enrichment_id</code> . Each chunk belongs to a document.
<code>document_enrichments</code>	Append-only record of each AI enrichment write, including model, skill ids, summary, payload, and timestamp.
<code>chunks_fts</code>	SQLite FTS5 table over chunk text and locator. Triggers keep it synchronized on chunk insert, update, and delete.
<code>invalidation_events</code>	Append-only audit trail for state transitions, with previous state, next state, reason, and timestamp.

The dense index is separate from SQLite. It is a Turbovec `IdMapIndex` using:

Setting	Value
Embedding profile	<code>local-hash-64</code>
Embedding dimension	<code>64</code>
Turbovec bit width	<code>4</code>
Index format	<code>tvim</code>
Dense-only hybrid threshold	<code>0.2</code>

The embedding is local and deterministic. The indexer tokenizes lower-case terms, hashes each token, updates a 64-dimensional signed vector, and normalizes it. There is no remote embedding model involved in this implementation.

7 Search Flow

The search API accepts:

Field	Behavior
<code>query</code>	Required, non-empty search text.
<code>limit</code>	Must be between 1 and 100.
<code>states</code>	Defaults to <code>active</code> .
<code>sourceTypes</code>	Optional filter such as <code>datasheet</code> , <code>book</code> , <code>website</code> , <code>repo_code</code> , <code>readme</code> , <code>media</code> , <code>image</code> , <code>spreadsheet</code> , or <code>text</code> .
<code>collection</code>	Optional exact collection filter.
<code>mode</code>	<code>hybrid</code> , <code>dense</code> , or <code>lexical</code> ; defaults to <code>hybrid</code> .

`lexical` mode uses SQLite FTS5 and BM25 ordering. It also boosts strict term matches and identifier-like terms such as register names, part numbers, and planted validation IDs so exact source facts are not buried by dense neighbors. `dense` mode searches Turbovec with an allowlist of eligible chunk IDs. `hybrid` mode runs both, drops weak dense-only candidates below 0.2, and combines the ranked lists with reciprocal rank fusion plus lexical relevance.

The result payload includes the chunk ID, document ID, title, source type, URI, locator, snippet, `chunkOrigin`, optional `enrichmentId`, fused score, optional dense and lexical scores, and citation metadata with the content SHA and snapshot path. That makes search results usable as source-grounded references instead of free-floating text, while still flagging whether a hit came from direct source extraction or AI enrichment.

`documentation.answer` builds on this search flow for the browser/API assisted-answer option instead of replacing source search. It answers a user question by retrieving the highest-value chunks, opening source documents for those chunks, and asking Codex to synthesize only from selected evidence. Small matched documents are eligible to be included as a whole; larger documents use the matched chunks and nearby source context. The answer response includes plaintext, semantic HTML for the panel, warnings when evidence is incomplete, citations, model name, and the search results used to build the prompt.

8 State And Invalidation

New and re-ingested documents become `active`. The non-active states are:

State	Intended meaning
<code>stale</code>	The source is outdated.
<code>superseded</code>	A newer source replaced this one.
<code>revoked</code>	The source is wrong or should no longer be trusted.

State	Intended meaning
<code>quarantined</code>	The source has trust, provenance, or extraction concerns.
<code>deleted</code>	The source was removed from normal active use.

Invalidation updates both the document row and its chunks, records an invalidation event, then rebuilds the Turbovec index. Removal is implemented as invalidation to `deleted`. The rows remain available for audit and explicit non-active searches.

9 API And Hook Map

9.1 Read Hooks

- `documentation.health`: GET `/health`. Returns health, archive root, schema, embedding profile, index path, portability flag, and `archiveLocality` invariant status.
- `documentation.stats`: GET `/stats`. Returns document and chunk counts plus portable paths, `archiveSize` storage/runtime totals, and `archiveLocality`.
- `documentation.portableManifest`: GET `/portable-manifest`. Returns the complete archive file manifest plus the same `archiveSize` totals.
- `documentation.documents.list`: GET `/documents`. Lists documents and defaults to active state.
- `documentation.documents.get`: GET `/documents/{id}`. Fetches one document with chunks and invalidation history.
- `documentation.search`: POST `/search`. Searches the archive and is exposed to plugin, UI, HTTP, automation, and voice.
- `documentation.answer`: UI/HTTP assisted-answer option. It searches the archive, opens source evidence, and returns a source-grounded AI answer when global AI control and Documentation AI enrichment are enabled. Codex-facing skills use `documentation.search` plus `documentation.documents.get` directly so Codex can inspect sources itself.

9.2 Write And External Hooks

- `documentation.ingest.path`: POST `/ingest/path`, safety write. Ingests allowed local files or directories.
- `documentation.ingest.url`: POST `/ingest/url`, safety external. Downloads external content, ingests a YouTube video with description, transcript, and keyframe artifacts, or expands a YouTube playlist into one media document per video.
- `documentation.ingest.text`: POST `/ingest/text`, safety write. Ingests copied text or manually supplied transcript text; only `text` is required.
- `documentation.invalidate`: POST `/invalidate`, safety write. Marks a document non-active with a reason.
- `documentation.remove`: DELETE `/documents/{id}`, safety write. Marks a document deleted.
- `documentation.rebuildIndex`: POST `/rebuild-index`, safety write. Rebuilds Turbovec from active SQLite chunks.
- `documentation.archive.export`: GET `/archive/export`, safety write. Streams a validated ZIP package to an allowed local output path without overwriting an existing file.

- `documentation.archive.import.merge`: `POST /archive/import/merge` or `POST /archive/import/merge/path`, `safety write`. Validates a ZIP package, imports missing stable documents, skips identical document IDs, reports document or URI conflicts, preserves invalidation events, and rebuilds the dense index.
- `documentation.archive.import.replace`: `POST /archive/import/replace` or `POST /archive/import/replace/path`, `safety destructive`. Requires the exact `REPLACE_DOCUMENTATION_ARCHIVE` confirmation token, validates the ZIP package, moves the current archive root aside, installs the imported root, and rebuilds the dense index.

The `DocumentationClient` defaults to `http://127.0.0.1:7820`, a 30 minute timeout, and an 8 MiB maximum response body. `CLOUDX_DOCUMENTATION_TIMEOUT_MS` sets the indexer request and AI enrichment timeout up to 12 hours for large PDF extraction or long media imports. `CLOUDX_DOCUMENTATION_RESPONSE_MAX_BYTES` sets the maximum indexer response size, and `CLOUDX_DOCUMENTATION_UPLOAD_MAX_BYTES` sets the browser documentation upload cap. Failed service responses are converted into plain error messages when the response includes `detail`, `message`, or `error`.

Binary upload is exposed through HTTP rather than a plugin hook because hook inputs are JSON-shaped. The browser route is `POST /api/documentation/upload` with an `application/octet-stream` body and metadata in query parameters. The indexer route is `POST /ingest/upload` with multipart form data. When AI enrichment is enabled, the same upload bytes are also passed to the enrichment service so media uploads can be transcribed and sampled before derived chunks are written by `POST /documents/{id}/enrich`.

Browser archive controls use `GET /api/documentation/archive/export` for downloads and `POST /api/documentation/archive/import/{merge|replace}` for ZIP uploads. The server forwards archive imports to the indexer as multipart uploads and forwards archive exports as streamed ZIP responses.

10 UI Workflow

In CloudX, create a Documentation tab. The panel gives a compact operational view:

1. Refresh archive stats and active document list.
2. Ask an AI-assisted archive question by default when AI assistance is enabled.
3. Switch to manual search when the user needs raw matching chunks or AI assistance is disabled.
4. Search active knowledge with the default hybrid retrieval path and optionally narrow by collection.
5. Queue knowledge imports from uploaded files, allowed local paths, URLs, YouTube videos/playlists, or direct text. Imports run one at a time so long YouTube/video work does not overlap.
6. Leave title and collection blank when the indexer should autodetect them from uploaded filenames, local folders, URL hosts, playlist metadata, or the first text line.
7. Track queued, running, complete, and failed imports in the Import Queue. File uploads report real byte progress from browser upload events; server-side extraction, URL download, transcript, keyframe, and enrichment work reports phase progress while the synchronous indexer request is running.
8. Read parallel progress channels for long media imports, including visual scan and audio transcript work when both are running.
9. Mark search results **stale**, **revoked**, **superseded**, or **quarantined**.
10. Remove a document from active search.
11. Open full source documents from search results or the Active Documents list to inspect chunks, transcript text, table Markdown, and extracted artifact metadata. Large sources auto-load more chunks and artifacts as the source viewer reaches the end.

12. Export the archive as a ZIP, import another archive by merge or confirmed replace, load the portable manifest, and rebuild the Turbovec index.

The UI defaults to active-document search, upload ingest, assisted-answer mode when AI assistance is available, and hybrid search under the hood. It shows AI-assistance state as a compact header icon. If AI is disabled, Answer mode is unavailable and the panel keeps manual search plus full source viewing. The Queue button snapshots the current form into a queued import and immediately clears the form so another source can be queued while the previous source is still downloading, transcribing, extracting, or enriching.

11 Default Rule And Skills

The plugin contributes this universal CloudX system rule automatically at server startup:

Rule	Purpose
<code>documentation-ingest-evidence</code>	Search active local archive records first. When adding evidence from a file, PDF, spreadsheet, image, URL, YouTube video, or playlist, ingest the original source so the full extractor runs; use text ingest only when no original source is available.

The plugin contributes seven CloudX system skills automatically at server startup:

Skill	Purpose
<code>documentation-search</code>	Mandatory local-first lookup for factual, research, recipe, recommendation, troubleshooting, summary, and source-grounded questions; when adding evidence, route original files, PDFs, spreadsheets, images, URLs, YouTube videos, and playlists through full ingest before answering from refreshed archive records.
<code>documentation-ingest</code>	Add original files, directories, websites, PDFs, images, YouTube videos/playlists, and copied text only when no retrievable original exists; prefer primary source URLs over search-result pages or low-trust mirrors.
<code>documentation-invalidate</code>	Find and invalidate stale, wrong, superseded, quarantined, or deleted sources.
<code>documentation-enrich-metadata</code>	Derive source-grounded metadata and searchable import-improvement notes.
<code>documentation-enrich-visuals</code>	Describe extracted tables, graphs, diagrams, screenshots, flowcharts, and schematic artifacts without inventing visual facts, netlists, or connectivity maps.
<code>documentation-enrich-media</code>	Improve media imports using transcripts and selected keyframes when available.

Skill	Purpose
<code>documentation-archive-control</code>	Inspect health, stats, portable manifest, export/import ZIP packages, and rebuild status.

The operational skills tell Codex to read `CLOUDX_DOCUMENTATION_URL` first and prefer the bundled helper over raw endpoint calls. The search skill still routes Codex away from `documentation.answer`, because Codex should inspect retrieved sources itself. It also instructs Codex to prefer active local archive evidence, use built-in web search only when local evidence is absent or insufficient, prefer official, vendor, spec, peer-reviewed, government, or reputable-news sources depending on the domain, ingest useful online sources back into the archive, and rerun local search before answering. When Codex tabs are launched from CloudX, the server exports that URL to child processes.

The operational documentation skills also bundle `scripts/cloudx-doc.mjs`, a small helper that wraps search, schematic search, open, list, ingest-url, ingest-path, ingest-text, invalidate, remove, health, stats, manifest, rebuild, archive export, archive merge import, and archive replace import calls so Codex can use short commands instead of handwritten curl/JSON/NDJSON requests. Archive helpers are `node "$DOC" export --output documentation-archive.zip`, `node "$DOC" import-merge documentation-archive.zip`, and `node "$DOC" import-replace documentation-archive.zip --confirm REPLACE_DOCUMENTATION_ARCHIVE`. Code-heavy path and URL helper ingests accept `--acceptGeneratedCodeDocumentation` and optional `--retainRawCodeArtifacts`; without the review flag the indexer rejects code-heavy input before writing documents. The enrichment skills stay instruction-only because they define model output behavior rather than endpoint operation.

The injection path is generic. Plugins expose `ruleContributions` and `skillContributions`, `syncPluginContributions` writes them into the CloudX `system-rules/` and `system-skills/` catalogs, and each Codex home overlay materializes all system rules into `AGENTS.override.md` plus all system skills under `skills/cloudx-system/`. A future plugin should follow the same pattern: use IDs prefixed with the plugin ID, provide a single-line rule text or complete `SKILL.md` body through the contribution, add bundled helper files when they reduce repeated tool boilerplate, and let the catalog and overlay handle availability for every Codex tab. Documentation enrichment deliberately uses this same catalog lookup; adding another plugin-owned enrichment pipeline should mean adding plugin skills, optional helper files, and a service that reads configured skill ids, not adding hardcoded extraction instructions.

12 Setup And Operations

Install and start the indexer:

```
npm run documentation:setup
npm run documentation:start
```

The equivalent explicit start command is:

```
CLOUDX_DOCUMENTATION_DATA_DIR=.cloudx/documentation \
services/documentation-indexer/.venv/bin/cloudx-documentation-indexer \
--host 127.0.0.1 --port 7820
```

CloudX defaults to:

```
CLOUDX_DOCUMENTATION_URL=http://127.0.0.1:7820
CLOUDX_DOCUMENTATION_DATA_DIR=.cloudx/documentation
```

First-class export:

```
DOC="$CLOUDX_RULES_SKILLS_DIR/system-skills/documentation-archive-control/scripts/cloudx-doc.mjs"
node "$DOC" export --output documentation-archive.zip
```

First-class import:

```
DOC="$CLOUDX_RULES_SKILLS_DIR/system-skills/documentation-archive-control/scripts/cloudx-doc.mjs"
node "$DOC" import-merge documentation-archive.zip
node "$DOC" import-replace documentation-archive.zip --confirm REPLACE_DOCUMENTATION_ARCHIVE
```

Raw indexer archive endpoints:

```
curl -L http://127.0.0.1:7820/archive/export -o documentation-archive.zip
curl -F file=@documentation-archive.zip \
  -F confirmation=REPLACE_DOCUMENTATION_ARCHIVE \
  http://127.0.0.1:7820/archive/import/replace
curl -F file=@documentation-archive.zip \
  http://127.0.0.1:7820/archive/import/merge
```

Archive export uses a write lock, rebuilds the dense index first, copies non-database archive files, snapshots `catalog.sqlite` through SQLite online backup semantics, and writes a manifest with hashes for every ZIP member. Merge import validates the package, imports missing stable documents and related chunks/enrichments, skips identical document IDs, reports conflicts, preserves invalidation events, copies missing snapshot/artifact files, and rebuilds the dense index. Replace import is destructive: it requires the exact confirmation token, validates the package before touching the current root, moves the current archive aside as a backup, installs the imported root, and rebuilds the dense index.

Manual archive-root move:

1. Stop CloudX or otherwise stop writes to the documentation indexer.
2. Copy or move the complete archive directory, not individual files.
3. Update `CLOUDX_DOCUMENTATION_DATA_DIR` to the new directory and restart the indexer.
4. Verify `curl -sS http://127.0.0.1:7820/stats` reports `"archiveLocality": {"ok": true, ...}`.
5. Run `curl -sS -X POST http://127.0.0.1:7820/rebuild-index` when changing service versions or when you want to prove the dense index can be rebuilt from SQLite chunks.

13 Rendering This PDF

The Ubuntu installer installs the render toolchain used for this guide: Quarto 1.9.38, Pandoc, and TeX Live's XeLaTeX/LuaLaTeX engines. It also installs FFmpeg for documentation media keyframes. Regenerate the PDF from the Quarto source with:

```
npm run docs:memory:pdf
```

That command reads docs/MEMORY_PLUGIN_GUIDE.qmd and writes docs/MEMORY_PLUGIN_GUIDE.pdf.

14 What Is Tested

The indexer tests cover:

- Ingesting uploaded files, PDFs, local directories, image files, HTML websites, README files, generated vendor-code documentation, media transcripts, and AI-derived enrichment chunks.
- Extracting PDF table sidecars, rendered visual artifacts, normalized image artifacts, schematic description artifacts, spreadsheet sheet artifacts, and vendor-code manifests.
- Searching with hybrid retrieval, lexical-only behavior, strict identifier recall, and dense-only fallback above the configured threshold.
- Invalidation, deletion, portable manifest generation, archive export/import, archive reopen/restore behavior, FastAPI controls, and CLI help.

The server plugin tests cover hook registration, safety classes, local path policy enforcement, browser upload forwarding including generated-code flags, enrichment routing, assisted-answer routing, automatic plugin system-rule and system-skill contributions, direct Codex search-skill guidance, generic plugin contribution validation, and Codex overlay injection. The web panel tests cover upload ingest, visible text ingest, generated-code review acceptance, queued sequential imports, progress channels, assisted-answer mode, assisted-search loading state, sanitized HTML answer rendering, disabled-AI manual mode, source-viewer removal controls, progressive source auto-loading, full source viewing, and search hook calls. The client tests cover JSON POST behavior, multipart upload behavior, documentation upload progress events, enrichment POST behavior, browser upload request construction, and propagation of service error messages.

A realistic validation runner in `debug_tooling/documentation-validation/run_validation.py` downloads or reuses public Wikipedia pages, the TurboQuant arXiv PDF, the kernel.org Linux virtual memory manager book, GMSL2 datasheet PDFs, optional live YouTube videos with transcripts and keyframes, and a generated 2500-record mock corpus with planted facts. The latest required validation run used `--include-youtube`, ingested 9 documents into 5324 active chunks, produced 1885 table CSV files, 1885 table Markdown files, 596 rendered figure PNGs, 3134 YouTube keyframe artifacts, 2 YouTube keyframe indexes, and 2 YouTube metadata JSON files, then passed required hybrid, lexical, rebuild, stale-state, YouTube transcript recall, and YouTube artifact checks. Dense-only checks are retained as diagnostics because the current dense profile is a deterministic local hash embedding, not a semantic embedding model. See `debug_tooling/documentation-validation/README.md` before running it; generated corpus, archive, and summary files stay under the selected `--root` and should not be committed.

15 Current Boundaries

- The dense embedding model is deterministic local hashing, not a semantic model from a remote embedding service.
- Dense-only mode is diagnostic for this implementation. Use hybrid as the default user-facing recall mode.
- Dense search is restricted to active chunks because only active chunks are loaded into Turbovec.
- URL ingest supports only HTTP and HTTPS, performs direct fetches or YouTube media ingest, and does not crawl a site recursively.
- YouTube URL ingest stores timestamped transcript, metadata, selected slide-frame artifacts, and a visual sampling manifest. It runs ASR and the CPU-heavy visual scan concurrently, downloads a bounded video source once before local parallel slide scanning, and depends on live YouTube access, `yt-dlp`, FFmpeg, and the configured ASR backend. Faster-whisper is the default; Intel Arc acceleration requires the explicit `whisper-cpp` backend with a SYCL/OpenVINO-capable `whisper-cli` and visible `/dev/dri` GPU device access.
- Browser uploads and URL downloads are capped at 256 MiB.
- PDF graph and flowchart extraction preserves rendered visual artifacts, but it does not OCR labels or infer graph semantics.
- Schematic-aware ingest is Phase 1 only: it classifies schematic-like visual artifacts, writes searchable descriptions and schema-ready JSON, and leaves `analysisOutputs` empty. It does not run OCR, component detection, connectivity mapping, SPICE export, or netlist generation.
- AI enrichment is enabled by default as a Documentation plugin setting, but it still requires global AI control. If either setting is disabled, assisted answers and post-ingest AI spans are unavailable; manual source-text search and full source inspection still work.
- Path ingest is limited by CloudX path policy and the supported file suffix list.
- Backup is directory-level. Copying only `catalog.sqlite` or only `chunks.tvim` is incomplete.

16 Source Inspection Commands

These are the commands used to ground this guide in the repository:

```
rg --files | rg '(^README.md$|^docs/SETUP.md$|documentation|Documentation)'  
rg -n "DocumentationArchive|TURBOVEC|ingest_|search|invalidate" \  
  services/documentation-indexer/src/cloudx_documentation_indexer/archive.py  
rg -n "CLOUDX_DOCUMENTATION|/ingest|/search|/invalidate|rebuild-index" \  
  services/documentation-indexer/src/cloudx_documentation_indexer/main.py  
rg -n "documentation\.|defaultDocumentationSkills|PathPolicy" \  
  apps/server/src/plugins/DocumentationPlugin.ts \  
  apps/server/src/documentation/DocumentationClient.ts  
rg -n "Documentation|Manifest|Rebuild|Add Knowledge|CLOUDX_DOCUMENTATION" \  
  README.md docs/SETUP.md apps/web/src/ui/DocumentationPanel.tsx package.json  
rg -n "def test_|portable|rebuild|dense|hybrid|FastAPI" \  
  services/documentation-indexer/tests/test_archive.py  
rg -n "DocumentationPlugin|documentation\.search|automationSafety|callHook" \  
  apps/server/src/plugins/DocumentationPlugin.test.ts \  
  apps/web/src/ui/DocumentationPanel.test.ts \  
  apps/server/src/documentation/DocumentationClient.test.ts
```

```
PYTHONPATH=services/documentation-indexer/src \  
  services/documentation-indexer/.venv/bin/python \  
  debug_tooling/documentation-validation/run_validation.py \  
  --root /tmp/cloudx-documentation-validation --mock-count 2500  
npm run docs:memory:pdf
```